



Implementação e Desenvolvimento de um Módulo de marcação de Consultas Online no Frontend, utilizando React Native, para o Centro Médico Famor, em Luanda, Angola.

Adilson Domingos da Conceição - 2025586

Lisboa, 2026



Implementação e Desenvolvimento de um Módulo de marcação de Consultas Online no Frontend, utilizando React Native, para o Centro Médico Famor, em Luanda, Angola.

Este projeto é desenvolvido no âmbito do cumprimento dos objetivos do plano curricular do Mestrado em Desenvolvimento de Dispositivos Móveis, especificamente na disciplina de Programação Android, orientada pelo Professor Msc. Jorge Monge, no ano letivo 2025/2026.

Lisboa, 2026

Índice

1.	Introdução e Motivação	4
2.	Estado da Arte	5
3.	Arquitetura do Sistema	6
4.	Descrição das Tecnologias Utilizadas	7
5.	Modelação (diagramas UML).....	9
	Diagrama de Classe.....	9
	Diagrama de Caso de Uso.....	10
6.	Descrição da Implementação	11
7.	Integração Backend–Frontend	12
	7.1. Login e autenticação	12
	7.2. Listagem de especialidades e médicos.....	13
	7.3. Marcação e consulta de agendamentos	13
8.	Testes e Validação	14
	8.1. Testes de base de dados.....	16
9.	Limitações.....	16
10.	Trabalho Futuro.....	17
11.	Referências Bibliográficas	17

1. Introdução e Motivação

Nas últimas décadas, vivenciamos uma verdadeira revolução provocada pelas Tecnologias de Informação e Comunicação (TIC), que transformaram profundamente a forma como as organizações prestam serviços. No campo da saúde, a adoção de soluções digitais aproximou instituições e pacientes, tornando o atendimento mais ágil, eficiente e humano. As aplicações móveis têm um papel de destaque nesse contexto, ao proporcionarem acesso rápido, prático e disponível a qualquer momento aos serviços de saúde [1].

Com o aumento do uso dos dispositivos móveis, surgem sistemas cada vez mais pensados para o dia a dia das pessoas, trazendo comodidade e facilitando o acesso à informação. Como ressalta Sommerville [2], os sistemas de software são essenciais para modernizar as organizações, pois otimizam processos e melhoram significativamente a qualidade dos serviços oferecidos.

No Centro Médico Famar, em Luanda, a marcação de consultas ainda é feita de modo convencional, o que pode ser pouco prático para os utentes e tornar a gestão dos agendamentos mais complicada. Diante desse desafio, percebe-se a importância de adotar uma solução tecnológica que permita aos utentes agendar consultas de forma simples e acessível, diretamente pelo telemóvel.

Diante desse cenário, este projeto propõe desenvolver um módulo de marcação de consultas online, utilizando React Native, para facilitar o agendamento de consultas médicas no Centro Médico Famar. O objetivo é oferecer aos utentes uma plataforma digital prática, segura e acessível, tornando todo o processo mais simples e confortável. Além de atender aos objetivos do plano curricular do Mestrado em Desenvolvimento para Dispositivos Móveis, na disciplina de Android, sob orientação do Professor Mestre Jorge Monge, durante o ano letivo 2025/2026, o projeto busca responder a uma necessidade real do Centro Médico Famar, contribuindo para a modernização dos serviços e para uma experiência mais positiva dos utentes.

2. Estado da Arte

A transformação digital está a mudar a forma como cuidamos da nossa saúde. Hoje, a tecnologia permite que clínicas e hospitais estejam mais próximos das pessoas, tornando a gestão dos serviços mais eficiente e facilitando o acesso dos pacientes aos cuidados médicos. Um exemplo claro disso são os sistemas de marcação de consultas online, que evitam deslocações desnecessárias e tornam o agendamento muito mais simples e rápido.

Na área da saúde, essa evolução tecnológica facilita o acesso aos serviços médicos e melhora o dia a dia de quem precisa de atendimento. Os sistemas de marcação de consultas online, por exemplo, permitem que os utentes façam seus agendamentos de forma remota, sem filas ou esperas desnecessárias, tornando tudo mais prático e eficiente para todos.

No entanto, mesmo com todos esses avanços, muitas clínicas e centros médicos especialmente em países africanos em particular Angola ainda têm dificuldades para adoptar ferramentas digitais que facilitem o agendamento de consultas e o atendimento aos pacientes. Isso mostra como é importante criar soluções acessíveis e adaptadas à realidade de cada lugar, pensando sempre no bem-estar das pessoas.

Na área do desenvolvimento de aplicações móveis, tecnologias como React Native, Node.js e MongoDB têm permitido criar sistemas modernos, eficientes e fáceis de manter. É por isso que esta proposta busca, justamente, responder às necessidades do Centro Médico Famar, oferecendo uma solução digital para marcação de consultas online que seja simples, acessível e pensada para facilitar a vida de utentes e profissionais.

3. Arquitetura do Sistema

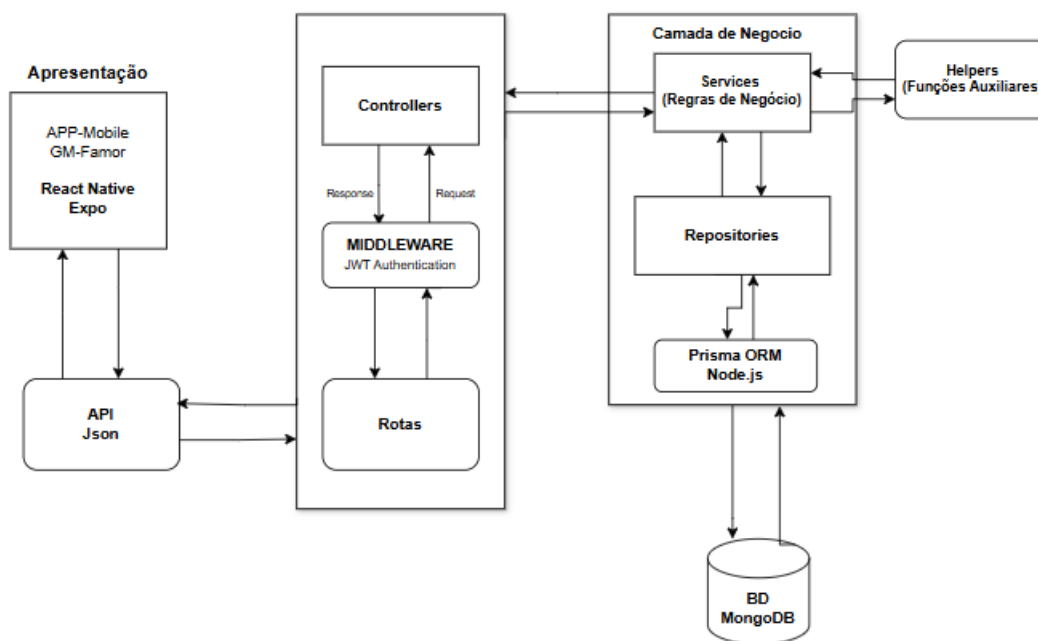


Figura 01: Arquitectura lógica do Sistema. **Fonte:** Autor, 2026.

Esta arquitetura foi pensada para tornar o sistema mais simples, organizado e fácil de manter ao longo do tempo. A intenção é dividir o sistema em várias partes, cada uma com o seu papel bem definido, facilitando o desenvolvimento e o cuidado contínuo da aplicação. Assim, cada componente contribui para que o processo de marcação de consultas médicas seja rápido, seguro e eficiente para todos os envolvidos.

Arquitetura define camada de apresentação, onde os utentes utilizam uma aplicação móvel intuitiva e fácil de usar. Por meio dela, pode registrar-se, fazer login, consultar especialidades médicas, ver os médicos disponíveis e marcar as suas consultas, através dos smartphones.

Para que esta solução funcione, a aplicação móvel conversa com o servidor por meio de uma API. Essa API recebe e processa os pedidos feitos pelos utilizadores, como marcações de consultas ou consultas de informações. Os caminhos (rotas) definidos na API garantem que cada pedido chegue à requisição solicitada, onde (controller) cuida de processar as operações de forma correta.

A forma da aplicação da segurança consiste em que antes de permitir acesso a qualquer área protegida, o sistema pede uma confirmação digital (token) que comprova que o utente está autorizado. Isso garante que apenas pessoas com permissão possam aceder a informações sensíveis.

Na camada de negócio ficam as regras principais do sistema, como gerir os utilizadores, consultar especialidades, verificar quais médicos estão disponíveis e fazer a marcação das consultas. Assim, as regras importantes ficam protegidas e organizadas, facilitando alterações e melhorias sempre que necessário.

Para guardar e recuperar todas as informações, existe a camada de persistência. Usamos ferramentas de persistência de dados, o Prisma ORM, para garantir que os dados estejam sempre disponíveis e bem organizados, conectando o funcionamento do sistema à base de dados.

Por fim, tudo o que é importante — como dados de utentes, médicos, especialidades e consultas — fica guardado com segurança no MongoDB, garantindo que as informações estejam sempre protegidas e acessíveis para o bom funcionamento do sistema.

4. Descrição das Tecnologias Utilizadas

O sistema de marcação de consultas online do Centro Médico Famar foi pensado para ser fiável, seguro e fácil de usar, recorrendo a tecnologias actuais que garantem um bom funcionamento tanto para quem gere como para quem utiliza o serviço.

1. React Native

O React Native é uma tecnologia que permite criar aplicações móveis para Android e iOS usando apenas um código base. Isto facilita o desenvolvimento, pois não é preciso fazer tudo duas vezes, tornando o processo mais rápido e eficiente.

2. Expo

O Expo funciona como apoio ao React Native, tornando mais simples o desenvolvimento, teste e publicação das aplicações móveis. Ajuda a usar funcionalidades do telemóvel sem grandes complicações técnicas.

3. Node.js

O Node.js serve para construir a parte do sistema que fica no servidor. Permite responder a vários pedidos ao mesmo tempo, garantindo que tudo funciona de forma rápida e sem atrasos.

4. Axios

O Axios é usado para enviar e receber informações entre a aplicação do utente e o servidor. Assim, os dados vão e vêm de forma organizada e segura.

5. JSON Web Token (JWT)

O JWT serve para garantir que cada utente entra com a sua conta e só tem acesso ao que lhe diz respeito. Ajuda a manter a segurança de todas as informações do sistema.

6. Prisma ORM

O Prisma ORM facilita o trabalho com a base de dados. Permite guardar, procurar e actualizar informações de forma mais simples, tornando todo o sistema mais organizado.

7. MongoDB

O MongoDB é onde ficam guardados todos os dados importantes, como informações dos utentes, médicos, especialidades e consultas. É uma base de dados flexível, ideal para sistemas que precisam crescer e adaptar-se.

8. Android

Durante o desenvolvimento da aplicação móvel, usou-se o **Android Emulator** como ambiente de testes. Esta ferramenta permitiu simular, em tempo real, o funcionamento da aplicação tal como aconteceria num telemóvel Android verdadeiro, evitando a necessidade de usar um aparelho físico nas primeiras fases do desenvolvimento.

9. Gmail Service

O serviço Gmail permite enviar e-mails automáticos para os utentes, seja para confirmar registos, autenticar acessos ou avisar sobre consultas marcadas. Assim, todos ficam informados de forma rápida e prática.

Além disso, o sistema conta com helpers, que oferecem funções reutilizáveis para validação de dados, formatação de datas e envio de notificações. Por exemplo, depois de uma consulta ser marcada, o próprio sistema pode enviar ao utente um e-mail automático com todos os detalhes do agendamento.

Com essa estrutura, cada componente tem sua responsabilidade bem definida, tornando o sistema mais fácil de manter, evoluir e adaptar a novas necessidades, sempre pensando na melhor experiência para utentes e profissionais.

5. Modelação (diagramas UML)

Um diagrama de classes UML é uma técnica de modelação usada para descrever, de forma estruturada, as classes de um sistema, seus atributos e métodos, bem como os relacionamentos entre elas, permitindo transformar requisitos em um modelo de projeto que guia a implementação. [3].

Diagrama de Classe

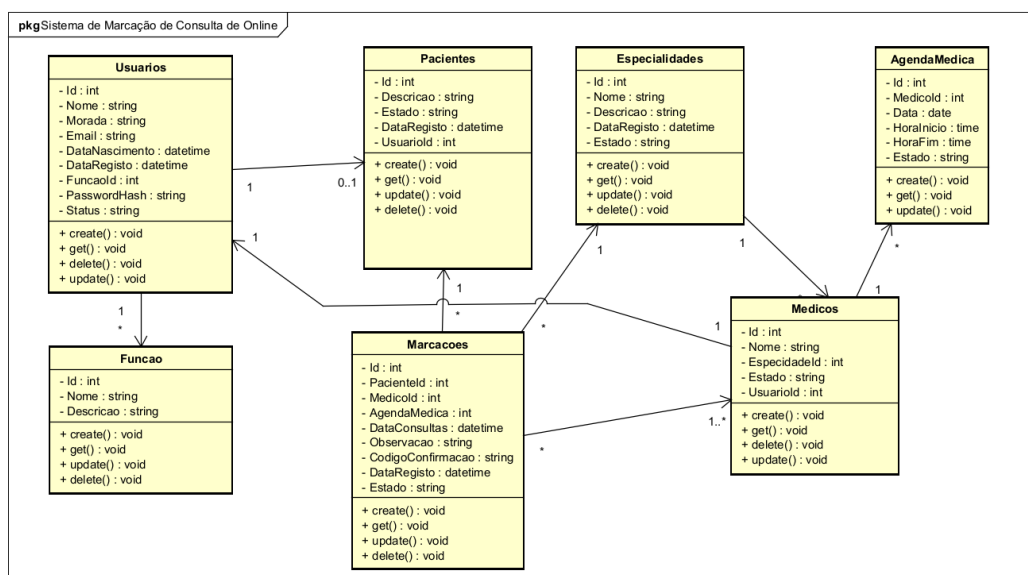


Figura 02: Diagrama de classe. Fonte: Autor, 2026.

No teu sistema de marcação para o Centro Médico FAMOR, o diagrama de classes é aplicado para modelar as principais entidades do domínio e suas relações. A modelação com UML não se limita à estrutura (classes); ela também apoia a análise do comportamento do sistema. Para um módulo de marcação online, é necessário compreender como o sistema reage às ações do utente (por exemplo:

selecionar especialidade, consultar disponibilidade por slots, submeter marcação e receber confirmação/cancelamento), garantindo que regras como prevenção de conflitos de agenda e transições de estado sejam corretamente definidas e implementadas.

Diagrama de Caso de Uso

O diagrama de casos de uso UML é uma notação usada para descrever funcionalidades do sistema sob a perspectiva dos utilizadores (atores) e do sistema, mostrando como atores interagem com o “Sistema” por meio de cenários nomeados (casos de uso). Ele explicita objetivos do sistema como “marcar consulta”, “cancelar marcação” e “consultar disponibilidade”. [4]

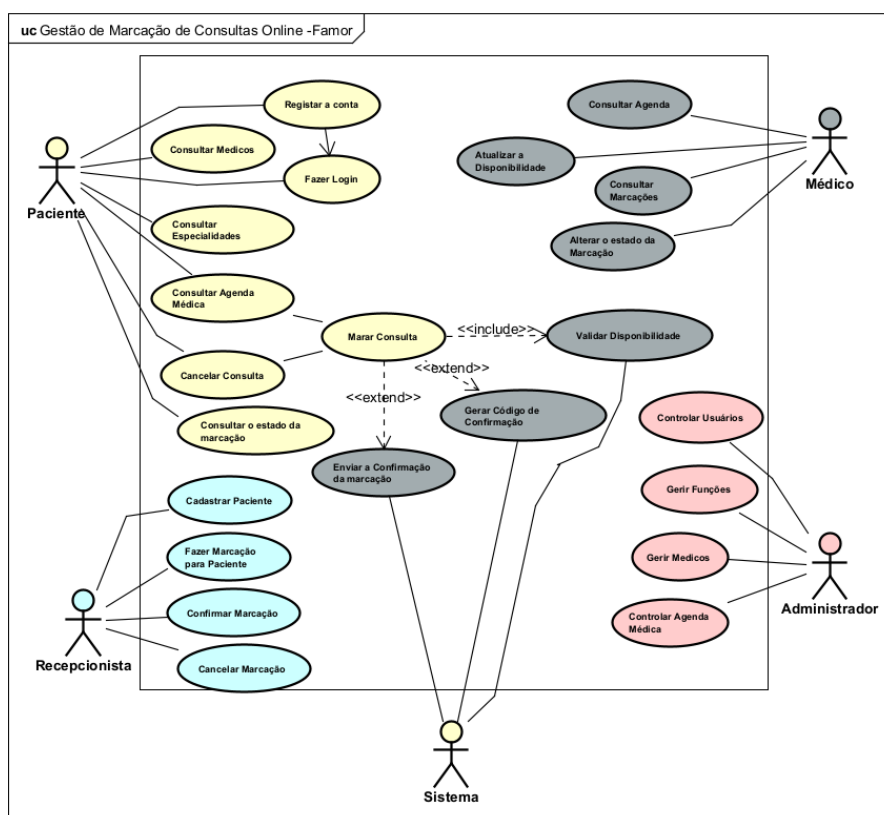


Figura 03: Diagrama de Caso de Uso. Fonte: Autor, 2026.

Embora o diagrama de classes descreva a estrutura (dados e relacionamentos), os casos de uso são usados para analisar o **comportamento** do sistema, isto é, as etapas e interações que ocorrem quando o ator executa uma atividade. Para um módulo de marcação online, analisar comportamento é essencial

para garantir que o fluxo do utilizador esteja correto, que o sistema valide regras (por exemplo, **validar disponibilidade e conflitos de agenda**) e que os estados da marcação (ex.: confirmada/cancelada) sejam tratados de forma consistente [5].

No teu diagrama, os atores e casos de uso refletem o comportamento do sistema de marcação:

- **Paciente:** consultar especialidades/médicos, consultar disponibilidade, marcar consulta, cancelar marcação e consultar estado, resultado da marcação.
- **Médico:** consultar agenda, atualizar informação da agenda (conforme o modelo) e interagir com a disponibilidade/consulta atribuída.
- **Rececionista:** gerir cadastro/contas (se aplicável) e operar fluxo de marcações (marcação/cancelamento) quando o processo envolver atendimento presencial ou apoio administrativo.
- **Administrador:** gerir utilizadores/perfis (ex.: gestão de perfis e ações administrativas) e controlar agenda/entidades do sistema.

Com isto garante que as funcionalidades do módulo (consulta, validação de disponibilidade, confirmação, cancelamento e consulta de estado) fiquem modeladas de forma coerente antes da implementação no frontend e no backoffice.

6. Descrição da Implementação

A implementação do sistema foi realizada seguindo uma arquitetura modular, dividida entre a aplicação móvel (frontend) e a API REST (backend). A aplicação móvel foi desenvolvida em React Native com Expo, permitindo criar interfaces responsivas e compatíveis com dispositivos Android. Toda a comunicação com o servidor é feita através de requisições HTTP utilizando Axios, com troca de dados em formato JSON.

No backend, a API foi construída em Node.js, organizada em rotas, controllers, serviços e repositórios. As rotas definem os pontos de acesso, os controllers tratam as requisições e validam dados, os serviços implementam as regras de negócio e os repositórios comunicam com a base de dados através do Prisma ORM. A base de dados utilizada é o MongoDB, escolhida pela sua flexibilidade e desempenho em aplicações móveis.

A autenticação é realizada com JWT, garantindo que apenas utilizadores autorizados acedem às funcionalidades protegidas. O sistema também integra helpers para validação, formatação de dados e envio de notificações por e-mail.

7. Integração Backend–Frontend

A integração entre o frontend e o backend segue o modelo cliente-servidor. A aplicação móvel envia requisições HTTP para a API, que responde com dados estruturados em JSON. Esta comunicação é síncrona e ocorre em todas as operações como demonstração:

Definição de factorização do código usando para chamar rotas específicas, deste modo (API/rotas.js), é a parte que permite várias rotas consumirem a mesma instrução.

```
function registerGetAliases(router, paths, handler) {  
  for (const path of paths) {  
    router.get(path, handler);  
  }  
}
```

Figura 01: Função registerGetAliases. Fonte: Autor, 2026.

7.1. Login e autenticação

```
function registerPublicRoutes(router, { usuarioController, senhaRecuperacaoController }) {  
  router.post('/usuarios', (req, res) => usuarioController.cadastrar(req, res));  
  router.post('/usuarios/confirmar', (req, res) => usuarioController.confirmarCadastro(req, res));  
  router.post('/login', (req, res) => usuarioController.login(req, res));  
  router.post('/senha/recuperar', (req, res) => senhaRecuperacaoController.solicitarRecuperacao(req, res));  
  router.post('/senha/resetar', (req, res) => senhaRecuperacaoController.resetarSenha(req, res));  
}  
  
function registerUsuarioRoutes(router, usuarioController) {  
  router.get('/usuarios', (req, res) => usuarioController.listar(req, res));  
  router.get('/usuarios/listagemusuario', (req, res) => usuarioController.listarSimples(req, res));  
  router.post('/usuarios/cadastro-admin', (req, res) => usuarioController.cadastrarDiretoPorAdministrador(req, res));  
  router.get('/usuarioperfil', (req, res) => usuarioController.obterPerfil(req, res));  
  router.get('/usuarioperfil/:id', (req, res) => usuarioController.obterPerfil(req, res));  
  router.get('/usuarios/pesquisar', (req, res) => usuarioController.pesquisar(req, res));  
  router.put('/usuarios/:id', (req, res) => usuarioController.atualizar(req, res));  
  router.delete('/usuarios/:id', (req, res) => usuarioController.deletar(req, res));  
}
```

Figura 02: Rota de login, serve para autenticar o utilizador. Fonte: Autor, 2026.

```
# JWT autenticação  
# jwtSecret é a chave secreta usada para assinar e verificar os tokens JWT.  
# É importante que seja uma chave aleatória e complexa, e não deve ser compartilhada entre aplicativos.  
# A chave secreta deve ser armazenada em um lugar seguro e não deve ser exibida para usuários finais.  
JWT_SECRET="d03b77add594c78fff5695942e7d398a2dbe13964ee632c05e44e42346a91d07"  
JWT_EXPIRES_IN="19y"
```

Figura 03: Este token é usado nas rotas protegidas. Fonte: Autor, 2026.

7.2. Listagem de especialidades e médicos

```
44  async listarComMedicos(req, res) {  
45    try {  
46      const especialidades = await this.especialidadeService.listarComMedicos();  
47      return res.json(especialidades);  
48    } catch (error) {  
49      console.error('Erro ao listar especialidades com médicos:', error);  
50      return res.status(500).json({ error: 'Erro ao listar especialidades com médicos.' });  
51    }  
52  }
```

Figura 04: A rota de listagem de especialidades com médicos. Fonte: Autor, 2026.

7.3. Marcação e consulta de agendamentos

A funcionalidade de marcação e consulta de agendamentos foi organizada em diferentes ficheiros, seguindo a separação de responsabilidades. O ficheiro **API/rotas.js** define as rotas responsáveis por receber os pedidos da aplicação móvel. Estas rotas encaminham as requisições para o **MarcacaoController.js**, que coordena o fluxo da operação. Em seguida, o **marcacaoService.js** aplica as regras de negócio, como validação de disponibilidade, criação da marcação e consulta dos agendamentos. O **marcacaoRepository.js** é responsável pela comunicação com a base de dados, enquanto o **marcacaoHelpers.js** contém funções auxiliares reutilizadas no processo. Assim, a rota funciona como ponto de entrada, mas depende das camadas de controller, service, repository e helpers para concluir correctamente as operações de marcação e consulta.

```
6  async criar({ pacienteId, medicoId, agendaMedicaId, dataConsultas, observacao, codigoConfirmacao, dataRegi  
7    return this.prisma.marcacao.create({  
8      data: {  
9        pacienteId,  
10       medicoId,  
11       agendaMedicaId,  
12       dataConsultas,  
13       observacao,  
14       codigoConfirmacao,  
15       dataRegisto,  
16       estado,  
17     },  
18   });  
19 }  
20  
21 async obterPacientePorId(id) {  
22   return this.prisma.paciente.findUnique({  
23     where: { id },  
24     select: {  
25       id: true,
```

```
139  registerEspecialidadeRoutes(protectedRouter, new EspecialidadeController(prisma));  
140  registerCrudRoutes(  
141    protectedRouter,  
142    ['/medico', '/medicos'],  
143    ['/medico/:id', '/medicos/:id'],  
144    new MedicoController(prisma)  
145  );
```

Figura 04: A rota de listagem de especialidades com médicos. Fonte: Autor, 2026.

8. Testes e Validação

A Figura 05, apresenta as principais telas do processo de autenticação da aplicação. Inicialmente, o utilizador efectua o registo através do preenchimento dos seus dados pessoais. Após a criação da conta, o sistema envia automaticamente um código de verificação para o endereço de correio electrónico informado. A ativação da conta apenas é concluída após a validação desse código, garantindo a autenticidade do utilizado.

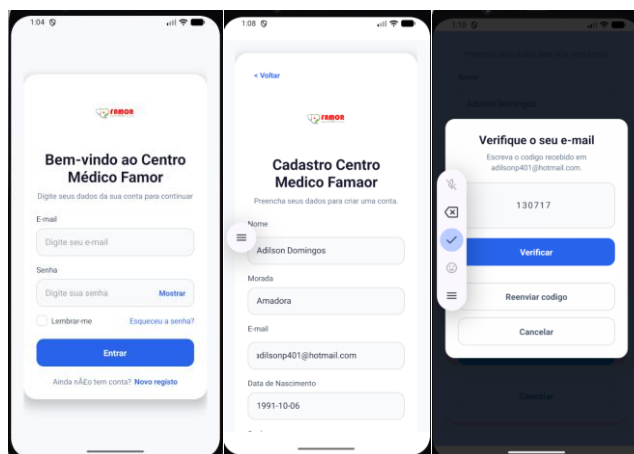


Figura 05: Telas de autenticação e validação de utilizadores da aplicação. **Fonte:** Autor, 2026.

A Figura 06, mostra as principais telas que o usuário encontra depois de fazer login no sistema. Primeiro, aparece a página de início de sessão, onde o utilizador coloca os seus dados para entrar. Depois da autenticação, o utente é levado para o painel principal da aplicação, onde pode rapidamente marcar consultas, ver as marcações já feitas, consultar as especialidades médicas disponíveis e aceder ao seu histórico de atendimentos.

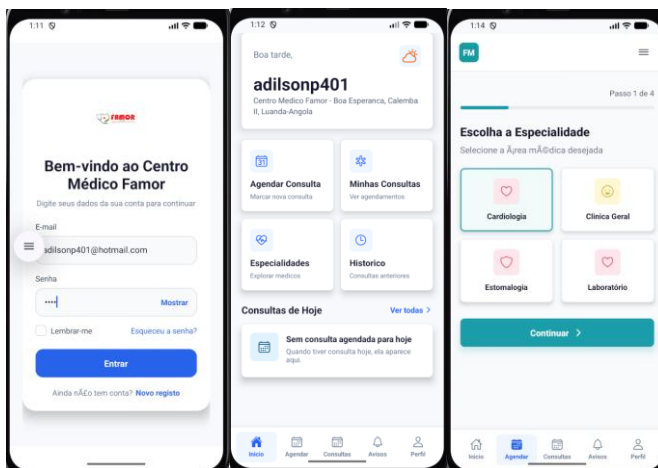


Figura 06 – Tela de início de sessão, painel principal e selecção de especialidades médicas. **Fonte:** Autor, 2026.

A Figura 07, mostra como funciona o processo de marcação de consultas na aplicação. Primeiro, o utente escolhe o médico com base na especialidade que deseja. Depois, selecciona a data e o horário que lhe convêm e estejam disponíveis. Antes de finalizar, o sistema apresenta uma página de confirmação com todos os dados da marcação — médico, especialidade, data e hora — para que o utilizador possa confirmar se está tudo correcto antes de submeter o pedido.

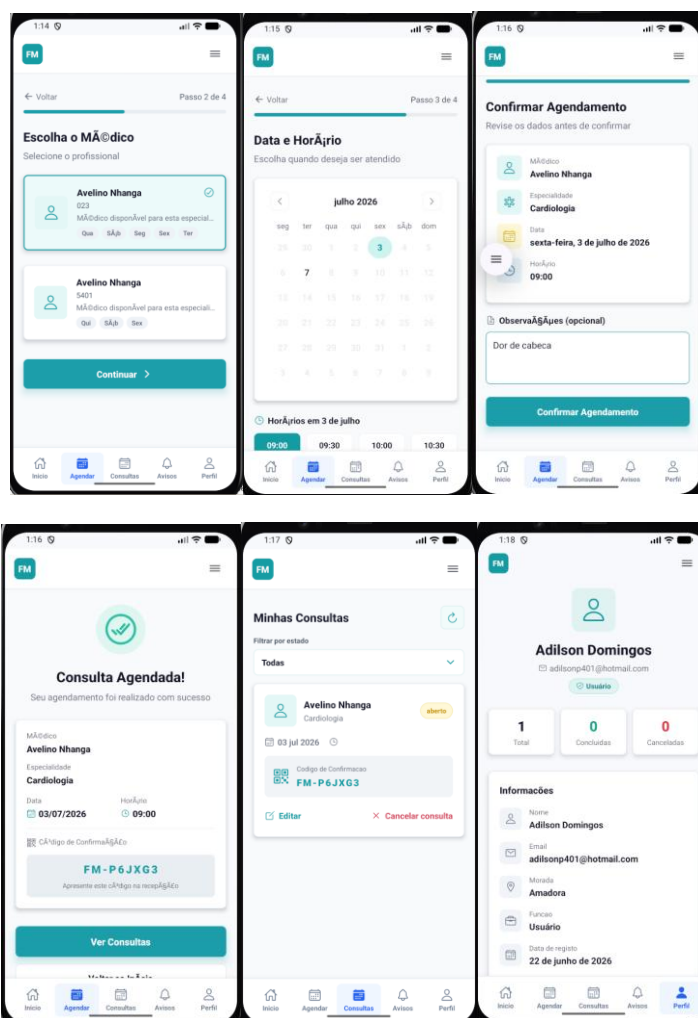


Figura 07 – Processo de marcação, confirmação e gestão de consultas médicas. **Fonte:** Autor, 2026.

8.1. Testes de base de dados

A validação dos testes de integração com o MongoDB será feita garantindo que os pedidos enviados à API resultam no registo correcto dos dados na base de dados. O processo inclui verificar se as operações de inserção, actualização e consulta guardam e devolvem os dados conforme o modelo definido, assegurando a ligação correcta entre entidades como utente, médico, agenda e marcações. Além disso, confirma-se que as consultas posteriores devolvem exactamente os dados que foram guardados. Conforme apresenta a figura 8.



	_id ObjectId	nome String	morada String	email String	datanascimento Date
1	ObjectId('6a2f8219db7aa91...	"root"	"root"	"root@famor.com"	2025-12-06T00:00:00.000
2	ObjectId('6a347c940745e07...	"rootadm"	"rt"	"root@gmail.com"	2025-12-06T00:00:00.000
3	ObjectId('6a347e240745e07...	"Adilsonp Domingos"	"Lisboa"	"adilsonconceicao678@gmail...	0199-06-20T00:36:45.000
4	ObjectId('6a39264dc05e9fb...	"Adilson Domingos"	"Amadora"	"adilsonp401@hotmail.com"	1991-10-06T00:00:00.000

Figura 08 – Validação dos dados persistidos na base de dados MongoDB. **Fonte:** Autor, 2026.

9. Limitações

Tendo o sistema funcional ainda e existem algumas limitações que devemos considerar importante para a conclusão destas funcionalidades:

- Ainda não possui um painel web de administração, o que impede a gestão automatizada de médicos, horários e especialidades.
- Depende de conexão à internet para sincronizar com o MongoDB; não há suporte a operações offline neste momento.
- As notificações aos utentes são enviadas apenas por e-mail. Ainda não há integração para o envio de mensagens por SMS ou por outras formas mais directas.
- A validação dos horários dos médicos ainda não considera situações mais complexas, como férias, ausências inesperadas ou outras interrupções na agenda.
- Neste momento, o sistema não permite pagamentos online, o que limita a activação de serviços pagos por parte dos utentes.

10. Trabalho Futuro

No futuro, queremos acrescentar à aplicação um módulo para rastreio do cancro da mama. Esta funcionalidade vai permitir que o utente use a câmara do telemóvel, juntamente com inteligência artificial, para ajudar a identificar, de forma inicial, alterações visuais que possam indicar necessidade de avaliação médica especializada.

O sistema pode analisar automaticamente as imagens captadas e mostrar ao usuário um relatório simples com sinais de risco. Se forem detectados sinais considerados suspeitos ou fora do normal, a aplicação pode sugerir que o utente faça exames complementares e, se necessário, encaminhá-lo para um centro especializado em oncologia.

Também pode ser feita uma ligação com os serviços nacionais de saúde, para que seja possível emitir automaticamente pedidos de consulta ou encaminhamentos médicos. Assim, estamos a contribuir para o diagnóstico mais cedo e para um melhor acompanhamento dos pacientes.

Esta funcionalidade é uma oportunidade para o sistema crescer além da simples marcação de consultas, tornando-se uma plataforma de apoio à prevenção, rastreio e acompanhamento de doenças oncológicas.

11. Referências Bibliográficas

- [1] K. Schwab, *The Fourth Industrial Revolution*. Geneva, Switzerland: World Economic Forum, 2016.
- [2] I. Sommerville, *Software Engineering*, 10th ed. Boston, MA, USA: Pearson, 2016.
- [3] OMG, *Unified Modeling Language (UML) Specification*. Object Management Group, documento de referência UML.
- [4] I. Jacobson, M. Christerson, P. Jonsson, and G. Övergaard, *Object-Oriented Software Engineering: A Use Case Driven Approach*, Addison-Wesley, 1992.
- [5] G. Booch, I. Rumbaugh, and J. Jacobson, *Unified Modeling Language User Guide*, Addison-Wesley, 1999.